Compte rendu : SAE 1.03 Réseaux

BHATT Piyush
SUSINI Charles BUT Informatique 1ère année 13 Mars 2025

Table des matières

1) Projet..... 2
2) Méthode d'installation..... 2
2.a) Création de la machine virtuelle..... 2
2.b) Configuration de la machine virtuelle..... 3
2.c) Installation Netkit-ng..... 4
3) Difficultés rencontrés..... 5
4) Création des machines netkit-ng..... 6
5) Installation et configuration du serveur Apache2..... 9
6) Avantages et inconvénients du protocole HTTP..... 12
Avantages :..... 13
Inconvénient :..... 13
7) Wireshark..... 13
En conclusion..... 16
..... 16
Lexicographie :..... 16
Source :..... 17

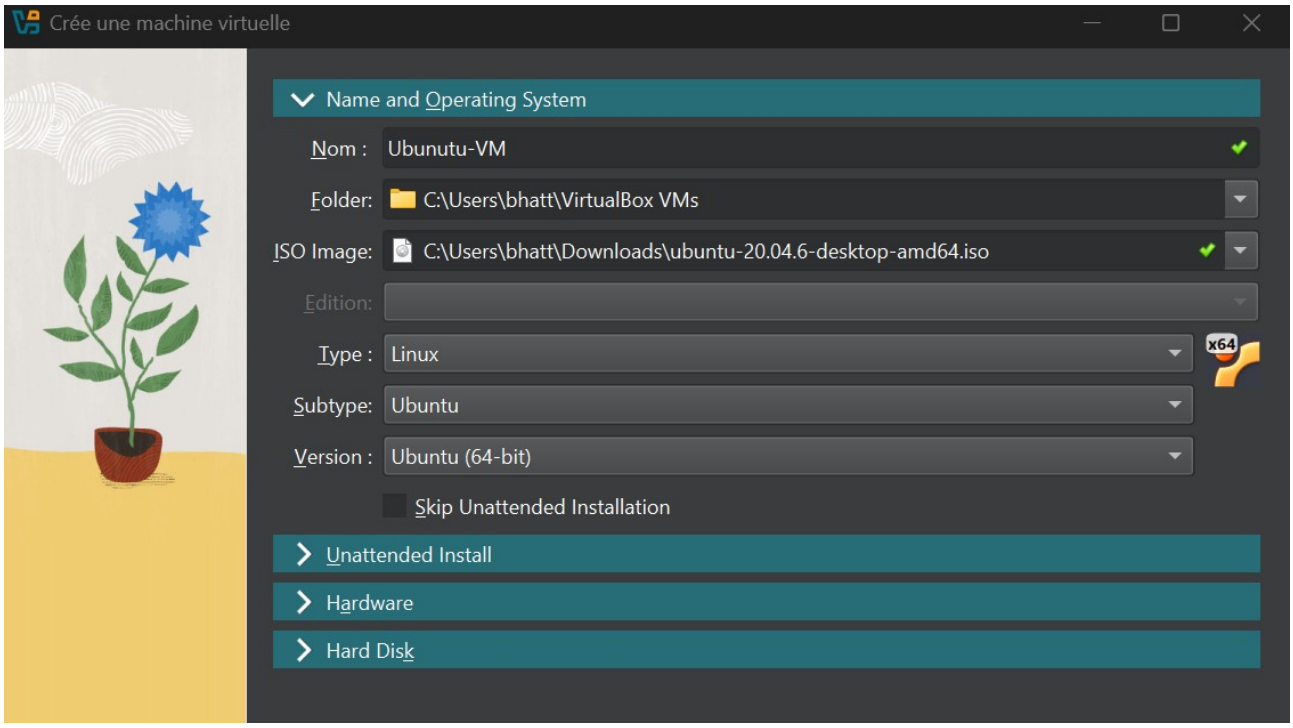
1) Projet

Dans le cadre de la SAE réseaux de cette année dans laquelle l'objectif était d'installer un serveur HTTP* sur une machine qui va jouer le rôle de serveur. Ensuite, s'assurer du bon fonctionnement du serveur HTTP en interrogeant celui-ci par des requêtes émises à partir d'une machine cliente M2 . Cette installation a pour but de nous familiariser avec le protocole HTTP, et un plan réseaux. Pour cette installation, on a choisi d'utiliser Ubuntu, une distribution* Linux, en machine virtuelle sur mon ordinateur portable équipé de Windows. Ubuntu est une distribution stable, largement utilisée dans le milieu professionnel.

2) Méthode d'installation

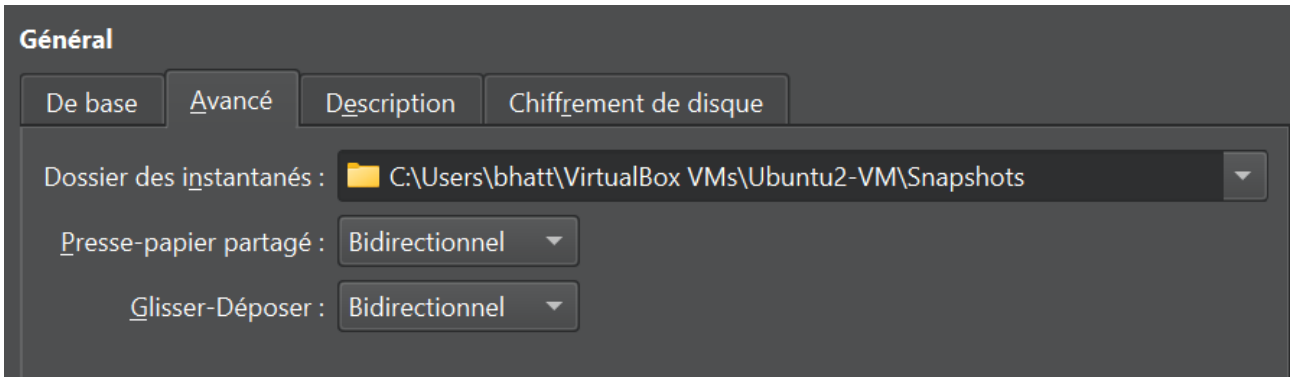
2.a) Création de la machine virtuelle

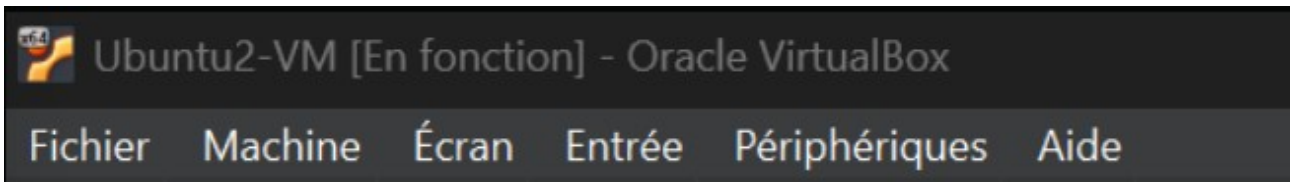
Avant de créer la machine virtuelle Ubuntu, on installe l'image ISO* d'Ubuntu https://releases.ubuntu.com/20.04/. D'ailleurs on utilise Oracle VirtualBox pour créer notre machine virtuelle, il nous a été introduit lors du premier semestre en python. Alors je clique sur « Nouvelle », je renseigne le nom qu'on veut donner à notre machine, dans mon cas c'est « Ubuntu-VM », puis on clique sur l'image ISO qu'on a installée et on le sélectionne. Dans la partie « Unattended Install », on choisit le username et le mot de passe. De plus, pour la mémoire vive on va dans la partie « Hardware » et on choisit 4 Mo, soit 4096 MB et on choisit 2 processeurs minimum pour ne pas avoir de soucis de latence. Et pour la taille on va dans la section « Hard Disk » et on choisit 30,00 Gio et finish pour valider. Et voilà que la machine virtuelle est créée.

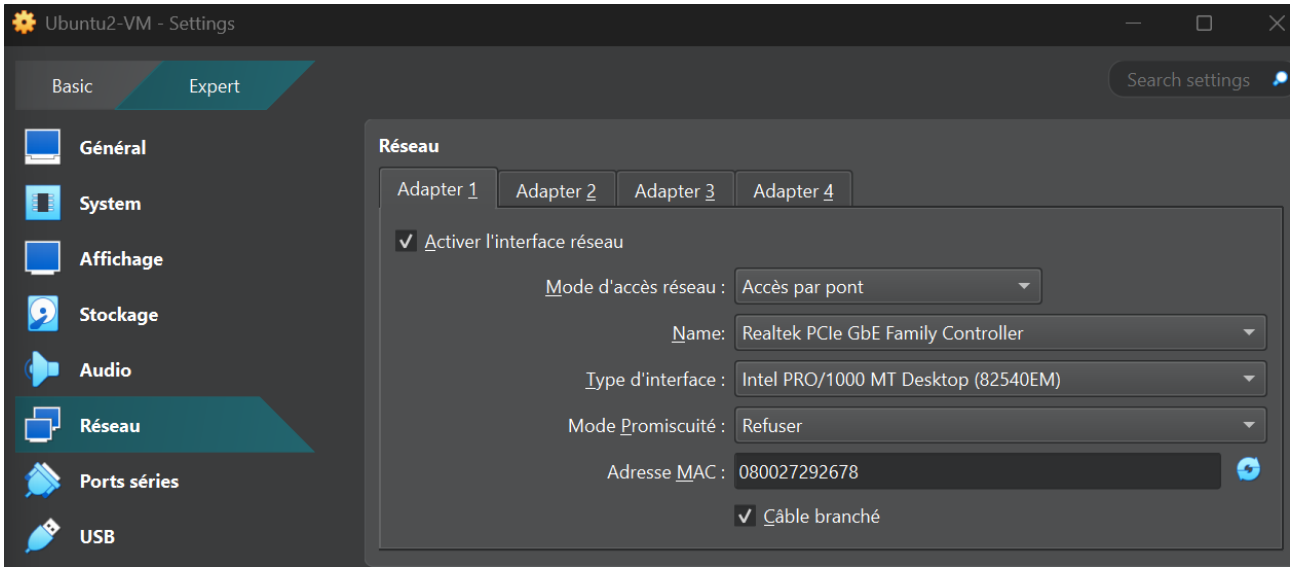


2.b) Configuration de la machine virtuelle

Tout d'abord, on fait la commande « sudo apt update && sudo apt upgrade -y » pour mettre à jour le système. Parfois il arrive qu'on a pas le sudo, donc on vérifie si on a accès a su – et on met le mot de passe. Puis on ajoute notre espace personnel avec les droits sudo, « usermod -aG sudo piyush », on vérifie si on a bien les droits mtn « groups piyush » si on voit notre espace suivi de sudo c'est bon. On fait « exit » pour quitter et on écrit la commande d'avant. Il se peut que le clavier et la langue soit en anglais, on peut la changer dans les paramètres. Dans notre cas, on voulait pouvoir copier coller de notre machine virtuelle à notre ordinateur portable. On est partie dans la configuration sur Oracle VirtualBox → Générale → Avancé, on voit presse papier partagé on met bidirectionnelle, de même pour glisser-déposer. Et on se connecte sur notre machine virtuelle, tout en haut on a périphérique, on clique dessus, puis Insérer l'image CV des additions invité et on écrit la commande « sudo apt install build-essential dkms linux-headers-$(uname -r) pour le plein écran et partage de fichiers. Et puis « cd/media/$USER/Vbox_Gas_* » et « sudo ./VboxLinuxAdditions.run » et on redémarre sudo reboot. Et faut aussi configurer le réseaux pour plus part pour ce faire on va dans configuration → réseau, pour l'adapter1 j'ai choisi de mettre accès par pont pour le mode d'accès réseaux car j'étais connecter en Ethernet, si c'étais par wifi il aurait fallu laisser en NAT, et j'active l'adapter2 en cochant la case « activer l'interface réseaux » et le mode d'accès je met réseaux privé hôte pour avoir une configuration réseaux isolé pour permettre aux machine Serveur et M2 du netkit de communiquer entre eux. Et avec ça on peut commencer l'installation du netkit en suivant la notice fournie.







2.c) Installation Netkit-ng

On suit la notice, en commençant par installer les fichiers fournis dans le lien suivant : https://igm.univ-mlv.fr/~quidelleur/Netkit-ng/. Après on va aller sur le root avec « sudo -i ». Puis on installe les paquetages suivants :
apt update
apt install uml-utilities lib32z1 xterm konsole wireshark net-tools
Puis on crée le répertoire Netkit-ng dans le répertoire /opt avec « mkdir /opt/Netkit-ng »

On vient copier dans le répertoire qu'on vient de crée les 3 fichiers qu'on a téléchargé avec « mv /home/piyush/Téléchargements/netkit* /opt/Netkit-ng » et on les décompresse dans notre répertoire Netkit avec :
«cd /opt/Netkit-ng
tar -xjSf /opt/Netkit-ng/netkit-ng-core-32-3.0.4.tar.bz2
tar -xjSf /opt/Netkit-ng/netkit-ng-filesystem_personnalise.tar.bz2
tar -xjSf /opt/Netkit-ng/netkit-ng-kernel-i386-K3.2-0.1.3.tar.bz2 » ça peut mettre un peu de temps.

Puis on va éditer le fichier .bashrc avec nano /root/.bashrc et on va a la fin du fichier et on met ces 3 lignes :
export NETKIT_HOME=/opt/Netkit-ng/netkit-ng
export PATH=$PATH:$NETKIT_HOME/bin export MANPATH=$MANPATH: $NETKIT_HOME/man, Ctrl X et on appuie sur O ou Y (si en anglais) et entrée pour valider. On va faire de même pour le fichier qui se trouve dans notre compte personnel avec « nano /home/piyush/.bashrc » et copié coller ces 3 phrases a la fin du fichier :
export NETKIT_HOME=/opt/Netkit-ng/netkit-ng
export PATH=$PATH:$NETKIT_HOME/bin export MANPATH=$MANPATH: $NETKIT_HOME/man, Ctrl X, O ou Y (si en anglais) et entrée pour valider.
On ouvre un nouveau terminal et on vérifie que l'installation est correcte :
sudo -i
cd /opt/Netkit-ng/netkit-ng
./check_configuration.sh, ce sera marqué activate si tout c'est bien passé et on fait « vstart pc » pour voir si netkit fonctionne, root@pc:~# devrait apparaître. On écrit halt pour fermer la machine virtuelle pc et rm pc.disk pour supprimer le fichier très volumineux créé par Netkit-ng. D'ailleurs pour pouvoir copier des informations de netkit on peut faire Ctrl + molette et select clipboard.

3) Difficultés rencontrés

On a eu plusieurs difficultés pour avoir une configuration correcte avant de commencer la SAE. La première c'est que je ne pouvais pas le faire en dual boot car j'avais déjà Debian d'installer, donc j'ai du utiliser une machine virtuelle alors que j'étais pas habitué. De plus, lors de l'installation de l'ISO d' Ubuntu j'ai pris la version la plus récente, une erreur car netkit n'est pas supporté dans les versions récentes. Donc j'ai perdu pas mal de temps à refaire toute la configuration de la machine virtuelle Ubuntu en prenant une version plus ancienne comme le 20.4. Et comme je n'avais pas l'habitude avec une machine virtuelle j'ai pris du temps à comprendre qu'il fallait que je configure le réseaux en accès par pont et que j'active un 2eme adaptateur..

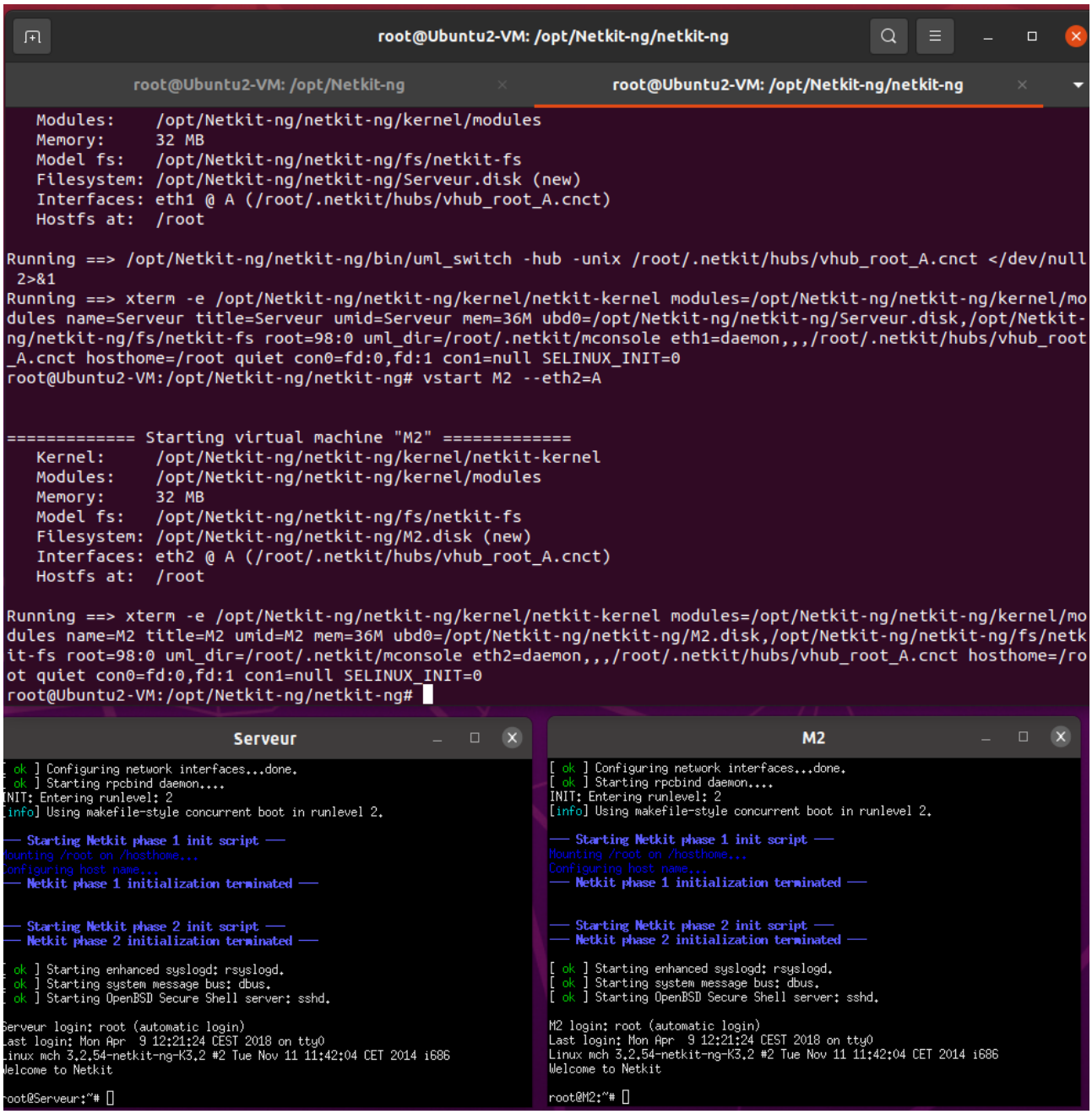
4) Création des machines netkit-ng

Pour construire le schéma sur la feuille de la SAE, j'ai crée 2 machine netkit M2 (client) et M1 (Serveur), en utilisant la commande « vstart Serveur --eth1=A » « vstart M2 --eth2=A », la on a crée nos 2 machines netkit, le serveur et le client (M2). Puis on attribue une adresse IP, « ifconfig eth1 192.168.56.10/24 up » la machine M2 et « ifconfig eth2 192.168.56.11/24 up » le Serveur. On fait ifconfig pour tester si ça était affecter. On les faits communiquer entre eux avec « ping 192.168.56.10 » Serveur → M2 et « ping 192.168.56.11 » M2 → Serveur.

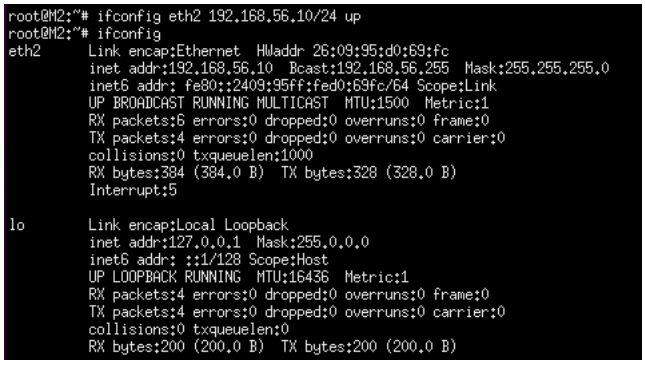
Schéma :

Compte rendu : SAE 1.03 Réseaux

BHATT Piyush
SUSINI Charles BUT Informatique 1ère année 13 Mars 2025



M2 :



Serveur :

```
root@Serveur:~# ifconfig eth1 192.168.56.11/24 up
root@Serveur:~# ifconfig
eth1      Link encap:Ethernet  HWaddr 62:bc:03:02:58:8c
          inet addr:192.168.56.11  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::60bc:3ff:fe02:588c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:238 (238.0 B)
          Interrupt:5

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:200 (200.0 B)  TX bytes:200 (200.0 B)
```

ping Serveur → M2 :

```
root@Serveur:~# ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_req=1 ttl=64 time=0.791 ms
64 bytes from 192.168.56.10: icmp_req=2 ttl=64 time=0.437 ms
64 bytes from 192.168.56.10: icmp_req=3 ttl=64 time=0.482 ms
64 bytes from 192.168.56.10: icmp_req=4 ttl=64 time=0.464 ms
64 bytes from 192.168.56.10: icmp_req=5 ttl=64 time=0.559 ms
64 bytes from 192.168.56.10: icmp_req=6 ttl=64 time=0.615 ms
64 bytes from 192.168.56.10: icmp_req=7 ttl=64 time=0.231 ms
64 bytes from 192.168.56.10: icmp_req=8 ttl=64 time=0.170 ms
64 bytes from 192.168.56.10: icmp_req=9 ttl=64 time=0.380 ms
64 bytes from 192.168.56.10: icmp_req=10 ttl=64 time=0.296 ms
64 bytes from 192.168.56.10: icmp_req=11 ttl=64 time=0.205 ms
64 bytes from 192.168.56.10: icmp_req=12 ttl=64 time=0.320 ms
64 bytes from 192.168.56.10: icmp_req=13 ttl=64 time=0.456 ms
64 bytes from 192.168.56.10: icmp_req=14 ttl=64 time=0.186 ms
^C
--- 192.168.56.10 ping statistics ---
14 packets transmitted, 14 received, 0% packet loss, time 13030ms
rtt min/avg/max/mdev = 0.170/0.399/0.791/0.174 ms
```

ping M2 → Serveur :

```
root@M2:~# ping 192.168.56.11
PING 192.168.56.11 (192.168.56.11) 56(84) bytes of data.
64 bytes from 192.168.56.11: icmp_req=1 ttl=64 time=0.968 ms
64 bytes from 192.168.56.11: icmp_req=2 ttl=64 time=0.858 ms
64 bytes from 192.168.56.11: icmp_req=3 ttl=64 time=0.315 ms
64 bytes from 192.168.56.11: icmp_req=4 ttl=64 time=0.459 ms
64 bytes from 192.168.56.11: icmp_req=5 ttl=64 time=0.540 ms
64 bytes from 192.168.56.11: icmp_req=6 ttl=64 time=0.409 ms
64 bytes from 192.168.56.11: icmp_req=7 ttl=64 time=0.413 ms
64 bytes from 192.168.56.11: icmp_req=8 ttl=64 time=0.434 ms
64 bytes from 192.168.56.11: icmp_req=9 ttl=64 time=0.458 ms
64 bytes from 192.168.56.11: icmp_req=10 ttl=64 time=0.193 ms
64 bytes from 192.168.56.11: icmp_req=11 ttl=64 time=0.198 ms
^C
--- 192.168.56.11 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10033ms
rtt min/avg/max/mdev = 0.193/0.476/0.968/0.232 ms
```

5) Installation et configuration du serveur Apache2

Apache2 est un serveur HTTP open-source largement utilisé pour héberger des sites web et des applications. Il permet de gérer les requêtes HTTP et d'envoyer des réponses aux clients (navigateurs). Pour l'installer sur Ubuntu on utilise la commande « sudo apt update » pour mettre à jour tout d'abord le système puis « sudo apt install apache2 -y ». Puis on peut vérifier si le serveur est bien actif avec « sudo systemctl status apache2 » et on devrait voir active(running) si tout c'est bien passé. Ensuite pour cette SAE il fallait créer 2 répertoire but1 et but2 « mkdir -p /var/www/but1 » et on remplace le 1 par 2 pour l'autre. Puis on devait créer un fichier index.html « echo "<html><body>ici la page de but1(2) </body></html>" > /var/www/but1/index.html ». De plus, on crée des fichiers de configuration des hôtes virtuels, pour le but1.conf on avait l'exemple sur le sujet.

"<VirtualHost *:80>

ServerAdmin webmaster@but1.inf

ServerName www.but1.inf

DocumentRoot /var/www/but1

ErrorLog /var/log/apache2/error_but1.log

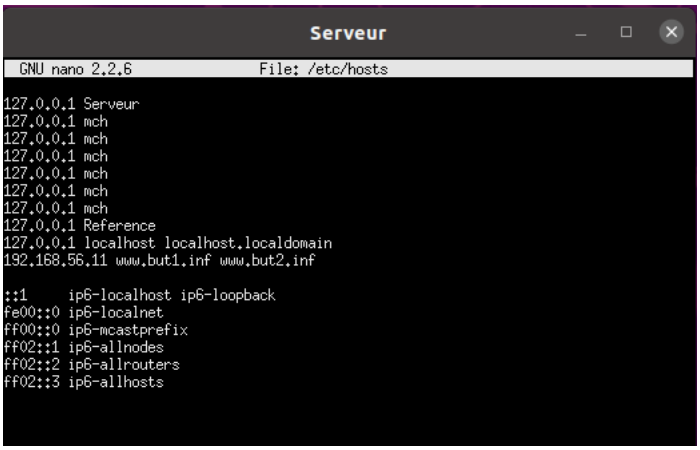
CustomLog /var/log/apache2/access_but1.log combined

</VirtualHost>"

On fait de même pour le but2.conf en remplaçant le 1 par 2. Enfin pour activer les sites et recharger Apache2 on fait « a2ensite but1(2).conf » et «service apache2 reload ». On modifie aussi le fichier /etc/hosts avec « nano /etc/hosts » et on ajoute cette ligne a la fin « 192.168.56.11 www.but1.inf www.but2.inf ».
On vérifie que www.but1.inf et www.but2.inf soit accessibles avec « curl http://www.but1.inf »

« curl http://www.but2.inf ».

La résolution de nom pour serveur :



ping du serveur but1 depuis M2 :

```
root@M2:~# ping www.but1.inf
PING www.but1.inf (192.168.56.11) 56(84) bytes of data.
64 bytes from www.but1.inf (192.168.56.11): icmp_req=1 ttl=64 time=0.605 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=2 ttl=64 time=0.257 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=3 ttl=64 time=0.521 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=4 ttl=64 time=0.171 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=5 ttl=64 time=0.499 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=6 ttl=64 time=0.200 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=7 ttl=64 time=0.184 ms
^C
--- www.but1.inf ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6031ms
rtt min/avg/max/mdev = 0.171/0.348/0.605/0.172 ms
root@M2:~#
```

Déclaration de l’existence du domaine but1:

```
Serveur
GNU nano 2.2.6    File: /etc/apache2/sites-available/but1.conf    Modified

<VirtualHost *: 80>
ServerAdmin webmaster@but1.inf
ServerName www.but1.inf
DocumentRoot /var/www/but1
ErrorLog /var/log/apache2/error_but1.log
CustomLog /var/log/apache2/access_but1.log combined
</VirtualHost>
```

Ping des liens but1 et but2 du M2 au Serveur:

```
root@M2:~# ping www.but1.inf
PING www.but1.inf (192.168.56.11) 56(84) bytes of data.
64 bytes from www.but1.inf (192.168.56.11): icmp_req=1 ttl=64 time=0.420 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=2 ttl=64 time=0.847 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=3 ttl=64 time=0.568 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=4 ttl=64 time=0.459 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=5 ttl=64 time=0.216 ms
^C
--- www.but1.inf ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4028ms
rtt min/avg/max/mdev = 0.216/0.502/0.847/0.206 ms
root@M2:~# ping www.but2.inf
PING www.but1.inf (192.168.56.11) 56(84) bytes of data.
64 bytes from www.but1.inf (192.168.56.11): icmp_req=1 ttl=64 time=0.296 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=2 ttl=64 time=0.593 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=3 ttl=64 time=0.341 ms
64 bytes from www.but1.inf (192.168.56.11): icmp_req=4 ttl=64 time=0.473 ms
^C
--- www.but1.inf ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 0.296/0.425/0.593/0.119 ms
```

M2 curl :

```
root@M2:~# curl http://www.but1.inf
<html><body>ici la page de but1</body></html>
root@M2:~# curl http://www.but2.inf
<html><body>ici la page de but2</body></html>
root@M2:~# curl http://192.168.56.11
<html><body>ici la page de but1</body></html>
```

La commande GET /index.html :

```
root@M2:~# telnet 192.168.56.11 80
Trying 192.168.56.11...
Connected to 192.168.56.11.
Escape character is '^]'.
GET Connection closed by foreign host.
root@M2:~# telnet 192.168.56.11 80
Trying 192.168.56.11...
Connected to 192.168.56.11.
Escape character is '^]'.
GET /index.html HTTP/1.1
host: www.but1.inf
If-Modified-SHTTP/1.1 408 Request Time-out
Date: Thu, 13 Mar 2025 21:45:13 GMT
Server: Apache/2.2.22 (Debian)
Vary: Accept-Encoding
Content-Length: 302
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>408 Request Time-out</title>
</head><body>
<h1>Request Time-out</h1>
<p>Server timeout waiting for the HTTP request from the client.</p>
<hr>
<address>Apache/2.2.22 (Debian) Server at 192.168.56.11 Port 80</address>
</body></html>
Connection closed by foreign host.
root@M2:~# ^C
root@M2:~# telnet 192.168.56.11 80
Trying 192.168.56.11...
Connected to 192.168.56.11.
Escape character is '^]'.
GET /index.html HTTP/1.1
Host: www.but1.inf
If-Modified-Since: Fri, 14 Oct 2022 12:38:50 GMT

HTTP/1.1 200 OK
Date: Thu, 13 Mar 2025 21:49:08 GMT
Server: Apache/2.2.22 (Debian)
Last-Modified: Thu, 13 Mar 2025 20:57:19 GMT
ETag: "16465-2e-6303f914bc9c0"
Accept-Ranges: bytes
Content-Length: 46
Vary: Accept-Encoding
Content-Type: text/html

<html><body>Ici la page de but1</body></html>

Connection closed by foreign host.
```

6) Avantages et inconvénients du protocole HTTP

HTTP de son nom « Hypertext Transfer Protocol » est un protocole couche application basé sur un modèle client-serveur. Il permet d’échanger des ressources (textes, images, vidéos, etc.) entre un

client HTTP (navigateur) et un serveur HTTP (par exemple : Apache). Son fonctionnement repose sur l'encapsulation et la désencapsulation des données à travers les différentes couches réseaux.

Client HTTP (Navigateur) → envoie une requête HTTP vers un serveur.

Serveur HTTP (Apache) → traite la requête et renvoie une réponse.
La communication se fait via TCP, généralement sur le port 80 comme vu en cours.

Avantages :

Il est simple et rapide, basé sur le modèle client-serveur, et utilise des requêtes textuelles ASCII qui sont faciles à comprendre. De plus il a une large adoption et compatibilité, il est utilisé par tous les navigateurs et compatible avec de nombreux systèmes, et basé sur TCP/IP, un standard universel. Enfin il offre une évolutivité avec HTTP/2 et HTTP/3. Le HTTP/2 optimise la gestion des requêtes en multipliant plusieurs requêtes en une seule connexion et le HTTP/3 utilise QUIC pour améliorer la vitesse et la sécurité.

Inconvénient :

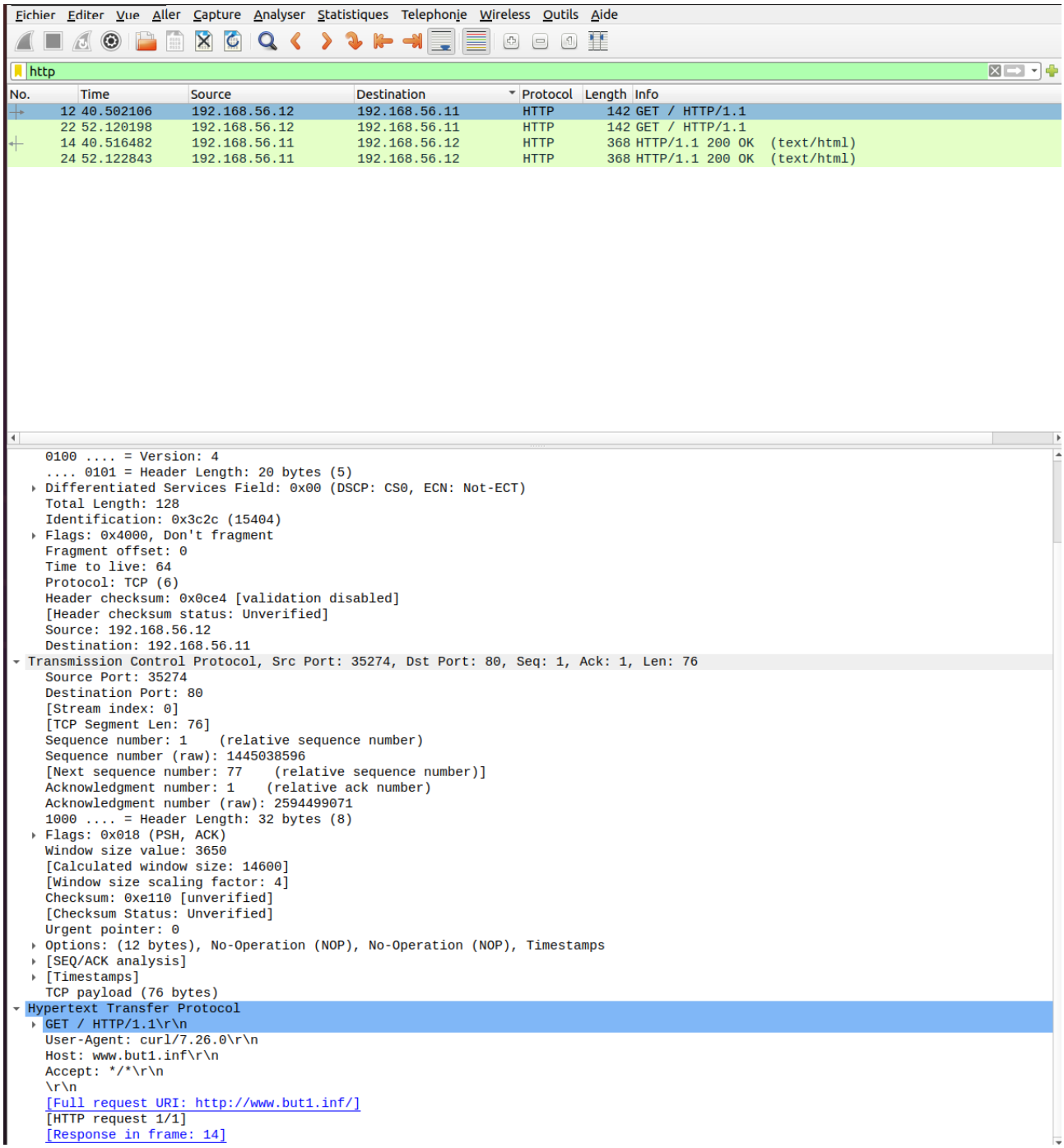
Tout d'abord, il n'y pas de sécurité intégrée comme on peut le voir avec des pages où en haut à gauche c'est marquer « cette page n'est pas sécurisée » avec le cadenas barré. C'est dû au fait que HTTP ne chiffre pas les données donc vulnérable aux attaques, la solution est d'utiliser HTTPS qui lui établit une connexion entre le navigateur et le serveur ce qui sécurise et chiffre avant de transférer des données. De plus, HTTP est un protocole stateless c'est-à- dire il ne mémorise pas les interactions précédentes. Ce qui oblige les applications web à utiliser des cookies ou des sessions pour garder des informations sur l'utilisateur. Enfin, la vulnérabilité aux attaques un pirate peut intercepter les communications ou bien manipuler les requêtes envoyées au serveur.

7) Wireshark

Je n'ai pas réussi a utiliser wireshark sur netkit, donc j'ai un peu dévié en utilisant la commande « tcpdump -i eth2 -w capture_http.pcap » pour enregistrer le trafic, puis de convertir le .pcap en base64 « base64 capture_http.pcap > capture_http.txt » puis de copier coller le contenu avec « cat capture_http.txt », le mettre sur ubuntu « nano capture_http_base64.txt » pour le décoder « base64 -d capture_http_base64.txt > capture_http.pcap » et enfin ouvrir wireshark « wireshark capture_http.pcap » et enfin avoir la réponses aux questions :

Le champs HTTP qui identifie l'hôte virtuel visé par le client est le champ Host. On y voit le « Host: www.but1.inf ». Le champ de la requête qui indique les langues supportées par le client HTTP est Accept-Language.

Le champ If-modified-Since dans la requête est la pour demander au serveur si une ressource a été modifiée après une certaine date. Le code de statut 200 Ok signifie que la requête a été traitée avec succès.
Enfin pour que le serveur renvoie toujours 200 OK, il faut que la date indiquée dans IF-Modified-Since soit antérieure à la dernière modification du fichier sur le serveur.


Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide
http
No. Time Source Destination Protocol Length Info
12 40.502106 192.168.56.12 192.168.56.11 HTTP 142 GET / HTTP/1.1
22 52.120198 192.168.56.12 192.168.56.11 HTTP 142 GET / HTTP/1.1
14 40.516482 192.168.56.11 192.168.56.12 HTTP 368 HTTP/1.1 200 OK (text/html)
24 52.122843 192.168.56.11 192.168.56.12 HTTP 368 HTTP/1.1 200 OK (text/html)
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 128
Identification: 0x3c2c (15404)
Flags: 0x4000, Don't fragment
Fragment offset: 0
Time to live: 64
Protocol: TCP (6)
Header checksum: 0x0ce4 [validation disabled]
[Header checksum status: Unverified]
Source: 192.168.56.12
Destination: 192.168.56.11
Transmission Control Protocol, Src Port: 35274, Dst Port: 80, Seq: 1, Ack: 1, Len: 76
Source Port: 35274
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 76]
Sequence number: 1 (relative sequence number)
Sequence number (raw): 1445038596
[Next sequence number: 77 (relative sequence number)]
Acknowledgment number: 1 (relative ack number)
Acknowledgment number (raw): 2594499071
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window size value: 3650
[Calculated window size: 14600]
[Window size scaling factor: 4]
Checksum: 0xe110 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[SEQ/ACK analysis]
[Timestamps]
TCP payload (76 bytes)
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
User-Agent: curl/7.26.0\r\n
Host: www.but1.inf\r\n
Accept: */*\r\n
\r\n
[Full request URI: http://www.but1.inf/]
[HTTP request 1/1]
[Response in frame: 14]

En conclusion

Ce projet avait pour objectif d'explorer le fonctionnement du protocole **HTTP** en installant et configurant un serveur **Apache2** sous Linux via **Netkit-ng**. Il s'agissait de mettre en place un environnement client-serveur, d'héberger plusieurs sites web sur une même machine et d'analyser les échanges réseau pour comprendre les mécanismes de communication entre un client et un serveur web.

Grâce à une approche méthodique, nous avons installé et configuré Apache, défini des hôtes virtuels, et assuré la résolution des noms de domaine via le fichier **/etc/hosts**. L'analyse du trafic HTTP avec **Wireshark** et l'exploitation de commandes comme `curl` et `telnet` ont permis d'observer les échanges et de vérifier la conformité des réponses du serveur.

Nous avons rencontré des défis liés à la configuration réseau et au transfert de fichiers entre Netkit et une machine virtuelle Ubuntu. Ces difficultés ont été surmontées en ajustant les paramètres réseau et en mettant en place des solutions comme l'utilisation de **SSH** et l'activation des interfaces réseau.

Ce projet nous a permis de développer des **compétences essentielles en administration système et en gestion des services web**, tout en renforçant notre compréhension des protocoles de communication. Il illustre concrètement l'importance d'une configuration rigoureuse et d'une analyse approfondie pour garantir le bon fonctionnement d'un serveur web.

Lexicographie :

Ci-dessous, voici les définitions de tous les termes techniques utilisé et marqué par un (*) :

• **HTTP** : HTTP de son nom « Hypertext Transfer Protocol » est un protocole couche application basé sur un modèle client-serveur. Il permet d'échanger des ressources (textes, images, vidéos, etc.) entre un client HTTP (navigateur) et un serveur HTTP (par exemple : Apache).

• **Apache** : le logiciel serveur Web qui traite les requêtes et diffuse les ressources et le contenu Web via HTTP.

• **Machine virtuelle** : c'est un environnement entièrement virtualisé qui fonctionne sur une machine physique. Elle exécute son propre système d'exploitation (OS) et bénéfice des mêmes équipement qu'une machine physique : CPU, mémoire RAM, disque dur et carte réseaux. Ce qui permet d'avoir par exemple windows et linux ubuntu en même temps contrairement au dual boot ou on aura qu'un seul OS.

• **Une distribution Linux** : parfois abrégé en distro appelée aussi distribution GNU/Linux lorsqu'elle contient les logiciels du projet GNU, est un ensemble cohérent de logiciels, la plupart étant des logiciels libres, assemblés autour du noyau Linux, et formant un système d'exploitation pleinement opérationnel.

• **ISO** : Un fichier ISO est, en termes simples, un format de fichier numérique reproduisant un CD, un DVD ou un BD physique. L'extension de fichier ISO ne se contente pas de stocker des fichiers et des dossiers : elle contient toutes les informations vitales du système de fichiers concernant la structure du disque.

Source :

Projet SAE :
https://elearning.univ-eiffel.fr/pluginfile.php/1008091/mod_resource/content/1/SAÉ - Installation_configuration d'un serveur HTTP-4.pdf

Installation de Netkit-ng:
https://elearning.univ-eiffel.fr/pluginfile.php/1008057/mod_resource/content/1/Notice_installation_Netkit-ng_2020-2021.pdf

Distribution Linux def:
https://fr.wikipedia.org/wiki/Distribution_Linux

Fichier ISO def:
https://www.winzip.com/fr/learn/file-formats/iso/#:~:text=Un fichier ISO est%2C en,concernant la structure du disque.

Wireshark:
https://elearning.univ-eiffel.fr/pluginfile.php/1008056/mod_resource/content/1/Notice_Wireshark.pdf

Commande Linux:
https://learn.microsoft.com/fr-fr/windows-hardware/manufacture/desktop/windows-and-gpt-faq?

view=windows-11

Apache2:
https://httpd.apache.org/docs/2.4/fr/

Définition HTTP :
https://elearning.univ-eiffel.fr/pluginfile.php/1008049/mod_resource/content/3/Cours1.pdf

HTTP: Différence HTTP et HTTPS
https://aws.amazon.com/fr/compare/the-difference-between-https-and-http/#:~:text=Comme son nom l'indique,avant de transférer des données.

HTTP/3 : QUIC
https://www.youtube.com/watch?v=1Wdzrpwqzsw